Unit Test Report

Online Exam Portal | Software Engineering Mini-Project, Part 3 | 10 November 2026

Student: Chiranthan V | SRN: PES2UG24CS138 | Section C

Release identifier: sprint2-final-2026-11-10 | Final submission report

Verified application commit: 25c23e04f4c810a0cbd1e68397aa3d63a9c4740c

Final main CI revision: 25c23e04f4c810a0cbd1e68397aa3d63a9c4740c. Backend, browser and performance runs below executed this merged revision.

The results below refer to these code versions. Earlier operational tests are listed separately with their dates.

1. Outcome and scope

PASS: 16 frontend component/function tests and 9 monitor regression tests. The supporting backend suite passed 101 tests on PostgreSQL, with no failures or skips. It includes API, integration and concurrency tests; these are not all isolated unit tests.

Suite	Passed	Failed / skipped	Environment
Frontend Vitest	16	0 / 0	Node 24.22.2; jsdom; 3 files
Monitor unittest	9	0 / 0	Python 3.11; clocks/mocks + local HTTP
Backend supporting regression	101	0 / 0	Python 3.11; PostgreSQL 16 CI
Earlier local backend cross-check	95	0 / 5	Windows Python 3.11.9; SQLite; original 100-test suite
Local frontend cross-check	16	0 / 0	Windows; existing installed Vitest 3.2.7

The earlier local run discovered 100 tests: 95 passed and five PostgreSQL-only cases were skipped. The recheck added a simultaneous-attempt-start test, so the current PostgreSQL run contains 101 passing tests, including six concurrency cases. The earlier local result is retained as a separate baseline, not relabelled as a run of the new test. Local monitor rerun: 9 passed in 3.721 seconds. PostgreSQL execution: 101 tests in 15.751 seconds. Results from different environments are not added together as unique cases.

2. Frontend unit/component results

Test file	Tests	Result
api.test.js	10	passed
attempt-recovery.test.js	3	passed
forms.test.jsx	3	passed

API tests check in-memory JWTs, CSRF/cookies, legacy attempt headers, concurrent refresh deduplication, session expiry, absent refresh cookie, partial-import errors, lockout, HTML error handling, network failure and pagination. Component tests cover distinct-option validation, stored text/server errors and correct-option selection. Recovery-age tests cover fresh/minute/hour values and unknown or reversed device-clock age.

3. Monitor regression results

All nine named cases passed: weighted complete-window accounting; overrun/missed slots; invalid URL/port/credentials/control characters; truncated response; TLS failure/socket timeout; concurrent summary-file nonoverwrite; invalid CLI/evidence reuse; invalid numeric configuration; actual local HTTP health/body/redirect behavior. No test disables TLS verification for real deployment evidence.

4. Supporting backend suite breakdown

Test module	Passed	Purpose
test_api	53	Authentication, administration and attempt APIs
test_assessment_acceptance	11	Results, reports and proctoring acceptance
test_assessment_integration	13	Grading and attempt lifecycle integration
test_frontend_delivery	3	Frontend delivery and security headers
test_injection_security	2	SQL injection and stored-script regressions
test_part3_concurrency	1	Simultaneous starts and winning-session ownership
test_performance_tools	2	Benchmark statistics and seed safety
test_postgres	5	PostgreSQL row locks and concurrent requests
test_scheduled_notifications	10	Notification scheduling, retries and deduplication
test_sprint2_migration_compatibility	1	Preservation of existing attempts during migration

The backend suite exercises authentication/lockout/RBAC/reset, roster imports, exam scheduling and selection, attempt binding and immutability, grading/reports/proctoring, notification outbox behavior, migration preservation and PostgreSQL races. Test dependencies are isolated fixture records and Django test databases; no real email receipt is implied.

5. Defects and limitations

The recheck found a gap in the test suite: duplicate-session behavior was checked sequentially, but simultaneous starts were not directly exercised. The new test uses two PostgreSQL transactions and a barrier inside the start path. It confirms one 201 response, one 409 response, exactly one attempt and start audit record, and resume access only for the winning session. The test passed; application code was not changed.

No assertion failures were found in the measured CI runs. The earlier confirmation-focus and sticky-timer defects were fixed in PR #60 and are covered by browser tests. Scoped team-lead acceptance is recorded separately from these automated results; no separate teammate or instructor sign-off is claimed. Local email tests check message generation; they do not establish delivery to a real mailbox.

Evidence and reproduction

Team-lead acceptance record: https://github.com/chiranthanv/online-exam-portal/pull/61#pullrequestreview-5838992122 . PR #64 approval/review is recorded through chiranthanv; a separate Harkirat/Dwijesh approval is not asserted.

Backend CI: https://github.com/chiranthanv/online-exam-portal/actions/runs/42783946825

Frontend/browser CI: https://github.com/chiranthanv/online-exam-portal/actions/runs/42783946813

Repository evidence: docs/testing/part3/evidence/2026-11-10-final/. The manifest records SHA-256 hashes. Downloaded archives were checked against GitHub's artifact digests before extraction. The selected JSON and text results are preserved without report screenshots. The original 100-test run remains in evidence/2026-11-06/; the earlier 101-test recheck remains in evidence/2026-11-06-recheck/.

Detailed coverage: coverage.json, coverage.xml and backend-coverage-original.zip (includes the interactive htmlcov/index.html report). Backend log lists every test; frontend JSON lists each assertion/scenario and actual status.

```
python -m unittest discover -s scripts -p 'test_monitor*.py' -v
coverage run backend/manage.py test portal.tests --verbosity 2
coverage report
cd frontend
npm ci
npm test -- --reporter=default --reporter=json --outputFile=unit-results.json
```

Run backend commands from repository root with disposable PostgreSQL DATABASE_URL and DEBUG=1. Use requirements-dev.txt. The exact Windows interpreter used for local cross-checks was C:/Users/91984/Downloads/online-exam-portal/.submission-demo/runtime/python311/python.exe; the isolated backend path was prepended explicitly because that embedded runtime has a preconfigured search path.